

Week3

Prerit Advani

June 2026

1 Bellman Expectation and Optimality Equations:

By the terms defined in the Previous week, we have:

$$v(S) = E[G_t | S_t = s]$$
$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

The value function can be decomposed into two parts using the linearity of expectation:

$$v(s) = E[R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \dots) | S_t = s]$$
$$\implies v(s) = E[R_{t+1} + \gamma G_{t+1} | S_t = s] = E[R_{t+1} + \gamma v(S_{t+1}) | S_t = s]$$
$$\implies v(s) = E[R_{t+1}] + \gamma E[v(S_{t+1}) | S_t = s] = \mathcal{R}_f + \sum_{s' \in S} P_{ss'} v(s')$$

Matrix Form:

$$V = R + \gamma PV$$

$$\begin{bmatrix} v(1) \\ \vdots \\ v(n) \end{bmatrix} = \begin{bmatrix} \mathcal{R}_1 \\ \vdots \\ \mathcal{R}_n \end{bmatrix} + \gamma \begin{bmatrix} \mathcal{P}_{11} & \dots & \mathcal{P}_{1n} \\ \vdots & \ddots & \vdots \\ \mathcal{P}_{n1} & \dots & \mathcal{P}_{nn} \end{bmatrix} \begin{bmatrix} v(1) \\ \vdots \\ v(n) \end{bmatrix}$$

- **v**: The value function vector, where each element $v(i)$ represents the value of state i .
- **r**: The immediate reward vector, where each $\mathcal{R}_i$ is the expected reward received in state i .
- γ : The discount factor, which determines the present value of future rewards.
- **P**: The transition probability matrix, where the element $\mathcal{P}_{ij}$ represents the probability of transitioning from state i to state j .

This is a linear equation and can be solved as follows:

$$(I - \gamma P)V = R \implies V = (I - \gamma P)^{-1}R$$

The computational complexity is $o(n^3)$. Hence, feasible only for small MRPs. The larger ones are solved using iterative methods like:

- Dynamic Programming
- Monte Carlo evaluation
- Temporal Differencing

When we have decide actions at each step using some policy, the process is known as Markov Decision Process.

We need to choose the action that maximizes reward at each step and this maximum value is known as the optimal value function of the state. $q_\pi(s, a)$ Denotes the reward starting at state $S_t = s$ with the action $A_t = a$ following the policy π after the first action.

$$\begin{aligned} q_\pi(s, a) &= \mathbb{E}_\pi[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a] \\ &= \mathcal{R}_s^a + \sum_{s' \in S} P_{ss'} v_\pi(s') \\ v_\pi(s) &= \sum_{a \in A} \pi(a|s) q_\pi(s, a) \\ &= \sum_{a \in A} \pi(a|s) (\mathcal{R}_s^a + \sum_{s' \in S} P_{ss'} v_\pi(s')) \\ q_\pi(s, a) &= \mathcal{R}_s^a + \sum_{s' \in S} P_{ss'} (\sum_{a \in A} \pi(a|s') q_\pi(s', a)) \end{aligned}$$

Matrix Form:

$$\begin{aligned} V_\pi &= \mathcal{R}_\pi + \gamma P_\pi V_\pi \\ \implies V_\pi &= (I - \gamma P_\pi)^{-1} \mathcal{R}_\pi \end{aligned}$$

Optimal Value Function:

$$\begin{aligned} v_*(s) &= \max_\pi v_\pi(s) \\ q_*(s, a) &= \max_\pi q_\pi(s, a) \end{aligned}$$

Partial Ordering over Policies:

A policy π is considered better than or equal to another policy π' (denoted as $\pi \geq \pi'$) if the state-value function of π is greater than or equal to that of π' for all states s :

$$\pi \geq \pi' \text{ if } v_\pi(s) \geq v_{\pi'}(s), \forall s$$

Theorem of Optimal Policy:

For any Markov Decision Process (MDP):

- Existence: There exists an optimal policy π_* that is better than or equal to all other policies ($\pi_* \geq \pi, \forall \pi$).

- Optimal Value Function: All optimal policies achieve the optimal state-value function, denoted as:

$$v_{\pi_*}(s) = v_*(s)$$

- Optimal Action-Value Function: All optimal policies achieve the optimal action-value function, denoted as:

$$q_{\pi_*}(s, a) = q_*(s, a)$$

Bellman Optimality Equations:

$$q_*(s, a) = \mathcal{R}_s^a + \sum_{s' \in S} P_{ss'} v_*(s')$$

$$v_*(s) = \max_a (q_*(s, a))$$

bellman Optimality equation is non linear and has no closed form in general. There are various iterative algorithms such as:

- Value Iteration
- Policy Iteration
- Q learning
- Sarsa

2 Policy Evaluation:

Aim: To evaluate a policy π

Solution (Pseudo Code):

- Start with all values as 0 i.e. $v_0(s) = 0 \forall s \in S$
- Update the value function for all states using the Bellman Equation:

$$v_{k+1}(s) = \sum_{a \in A} \pi(a|s) (\mathcal{R}_s^a + \sum_{s' \in S} P_{ss'} v_k(s'))$$

In matrix form,

$$V^{k+1} = \mathcal{R}^\pi + \gamma V^k P^\pi$$

- Keep iterating until $|V^{k+1} - V^k| < \epsilon$
- The sequence $V^0, V^1, V^2, \dots \rightarrow V^\pi$

3 Policy Iteration:

Aim: To improve the policy step by step and find the optimal policy.

Solution (Pseudo Code):

- Start with all values 0 and random Policy.
- Evaluate the value function at each state using policy evaluation algorithm mentioned above.
- Update the policy greedily i.e. at each state update the action to the action that maximizes the reward. If no state changes then the policy is stable.
- If policy is stable: stop, else: Go to step 1.

4 Value Iteration:

Aim: To improve the value function.

Solution (Pseudocode):

- Start with all values 0.
- Update the values as per the Bellman equation:

$$v(s) = \max_a \sum_{s'} P_{ss'}^a [R_{ss'}^a + \gamma v(s')]$$

- Stop if $\max(v(s)_{new} - v(s)_{old}) < \epsilon$. else Go to step 2.

5 Monte Carlo and Temporal differencing:

Although both of these methods are similar, model free and have the same aim, their answers often differ for finite set of sample. This is because of their different objectives. Monte Carlo minimizes the Mean Squared Error between the predicted value and the actual observed returns. While, Temporal Differencing builds an MDP that would explain the given samples. This can be demonstrated by a simple toy MDP as follows:

A 0 B 0 C 1

A 1

A 0 C 1

C 0

B 1

A 0 B 0

A 1 B 0

B 1 C 0

Monte Carlo:

$$v(A) = \frac{1 + 1 + 1 + 0 + 1}{5} = 4/5$$

$$v(B) = \frac{1 + 1 + 0 + 0 + 1}{5} = 3/5$$

$$v(C) = \frac{1 + 1 + 0 + 0}{4} = 2/4$$

Temporal Differencing ($\gamma = 1$):

Since C is the terminal state. Its value is calculated exactly like Monte Carlo.

$$v(C) = 1/2$$

From B:

- Leads to C with reward 1 (once)
- Leads to C with reward 0 (twice)
- Leads to termination/nothing with reward 1 (once)
- Leads to termination/nothing with reward 0 (once)

From A:

- Leads to B with reward 0 (twice)
- Leads to termination with reward 1 (twice)
- Leads to C with reward 0 (once)

$$v(B) = (2/5)(\frac{1+0}{2}) + (3/5)(\frac{1+0+0}{3}) = 2/5$$

Similarly,

$$v(A) = (3/5)(\frac{0+0+1}{3}) + (1/5) * 0 + (1/5) * 1 = 2/5$$